

scGPT + PaSCient Performance Analysis — Overview

Notebook: `exact_pascent/performance_analysis.ipynb` (Section 7) **Dataset:** `atlas_concatenated_v2_subset_UC_inflamed_baseline_v2_filter_1000_strat5`
Task: UC healthy-vs-inflamed binary classification at the patient level.

A concrete walkthrough with tiny numbers

Let's shrink the whole pipeline to something you can hold in your head, then map it back to the real setup.

The toy setup

- **20 patients total**, each with a `disease_binary` label (10 healthy, 10 diseased).
 - Each patient has between 3,000 and 8,000 single cells profiled.
 - We use **2 seeds** (in reality: 5) and **2 CV folds** (in reality: 5) and **2 rounds per fold** (in reality: 3).
-

Step 1 — Stratified downsampling (one-time preprocessing)

Cap each patient at 5,000 cells while keeping their cell-type mix.

Patient	Before	Cell types (before)	After 5k cap	Cell types (after)
P01 (Healthy)	8,000	60% T, 25% B, 15% Epi	5,000	60% T, 25% B, 15% Epi
P02 (Diseased)	3,000	50% T, 30% B, 20% Epi	3,000 (kept as-is)	50% T, 30% B, 20% Epi
P03 (Diseased)	7,500	70% T, 20% B, 10% Epi	5,000	70% T, 20% B, 10% Epi

Why: so no huge patient dominates and no tiny patient gets thrown away. Every patient contributes fairly to the model.

Step 2 — Split patients per seed (donors, not cells!)

For seed 0:

```
20 patients
|
+-- Test:      3 patients (15%)    <- locked away, never used in tr
|
+-- Train+Val: 17 patients
|
|   +- Fold 0:  train on 14,  validate on 3
```

```
+-- Fold 1:  train on 14,  validate on 3 (different 3)
+- ...
```

For **seed 1**: same procedure with a different random shuffle → a **different 3 test patients** and different fold splits.

Golden rule: a patient is never simultaneously in train and test. All cells from one patient stay together.

Step 3 — Train models

For each seed × each fold, train the model. Result:

```
seed_0/best_model_fold0.pt
seed_0/best_model_fold1.pt
seed_1/best_model_fold0.pt
seed_1/best_model_fold1.pt
```

Each .pt file contains: **scGPT backbone + LoRA adapters + patient-pooling head + classifier**.

All of the above is already done. The notebook only *evaluates* these saved checkpoints.

Step 4 — Evaluate (this is what Section 7 does)

For **seed 0** we compare "how well models score on patients they trained on" (Train + Val) vs "how well they score on truly unseen patients" (Test).

The inner loop, concretely — for one seed, on the Test set

We have 3 test patients (say **T1, T2, T3**) and 2 fold models. Each patient has 5,000 cells but the model can only process 2,000 cells at a time.

```
For each fold model (fold=0, fold=1):
  Load the model  ->  put on GPU
  For each round (r=0, r=1):
    Randomly pick 2,000 of T1's 5,000 cells -> forward pass -> prob
    Randomly pick 2,000 of T2's 5,000 cells -> forward pass -> prob
    Randomly pick 2,000 of T3's 5,000 cells -> forward pass -> prob
  Delete the model  ->  free GPU
```

For each patient we now have **2 folds × 2 rounds = 4 probability predictions**.

Averaging them into one score per patient

For patient T1:

```
fold 0, round 0:  P(disease) = 0.72
fold 0, round 1:  P(disease) = 0.78    <- different because different 2,
fold 1, round 0:  P(disease) = 0.65
fold 1, round 1:  P(disease) = 0.70
```

Ensemble mean: $P(\text{disease}) = 0.71$ <- this is the final prediction f
True label of T1 is "diseased" (1) → prediction 0.71 > 0.5 → **correct**.

Do the same for T2 and T3. You now have 3 ensembled predictions and 3 true labels → compute Balanced Accuracy, F1, AUROC, AUPRC.

Repeat for Train + Val

Same process but on the 17 Train + Val patients instead of the 3 Test patients. That gives you a second set of 4 metrics.

Repeat for seed 1

Different test patients, different fold models → another (Test, Train + Val) pair of metrics.

Step 5 — After all seeds, you have a table like this

Seed	Split	balAcc	F1	AUROC	AUPRC
0	Train + Val	0.96	0.95	0.98	0.97
0	Test	0.84	0.80	0.91	0.88
1	Train + Val	0.95	0.94	0.97	0.96
1	Test	0.87	0.83	0.93	0.90

Step 6 — The boxplot

For each metric (4 panels), draw two boxes: one from the Train + Val numbers, one from the Test numbers. Overlay each seed as a dot.

Balanced Accuracy		F1 Score		AUROC	A
+-----+	+-----+	+-----+	+-----+	...	...
Train	Test	Train	Test		
~0.95	~0.85	~0.94	~0.81		
+-----+	+-----+	+-----+	+-----+		

Delta = Train - Test = +0.10 (moderate overfitting gap)

Δ **close to 0** → model generalizes well. Δ **large** → model memorized training patients. This one number is the whole point of Section 7.

Why every design choice exists — in one line each

Choice	Purpose
Stratified 5k downsampling	Equal patient contribution, biology preserved

Donor-level split (not cell-level)	Prevents "patient identity" leakage
15% held-out test set	Honest, untouched evaluation
5-fold cross-validation	5 diverse models to ensemble; enables early stopping
Multiple seeds	Measures how much performance depends on which patients happen to land in test
Multiple rounds per fold	Averages away noise from picking only 2,000 of the 5,000 cells
Mean-softmax ensemble	Turns 15 noisy predictions into one confident one
Compare Train + Val vs Test	Measures the generalization gap Δ
4 metrics	Threshold + ranking + minority-class robustness

Real vs toy scaling

	Toy	Real
Patients	20	202 donors / 377 samples
Seeds	2	5
Folds	2	5
Rounds/fold	2	3
Passes per patient	4	15
Cells sampled per pass	2,000	2,000
Cells per patient after 5k cap	up to 5,000	up to 5,000
Effective cell coverage per patient	$4 \times 2000 = 8000$ (overlapping) ~98%	$15 \times 2000 \sim 99.95\%$

Same recipe — just scaled up.

Appendix — About embeddings

Are embeddings created for each fold $\times$ each round? Yes.

`model(batch, chunk_size=256)` runs the full stack per patient: scGPT backbone (with LoRA) $\rightarrow$ per-cell embeddings $\rightarrow$ aggregator $\rightarrow$ classifier logits. The embeddings are computed on every forward pass but are **not stored** — only the final softmax vector is kept.

Two independent things invalidate any caching:

1. **Different fold $\rightarrow$ different weights.** Each fold checkpoint has different LoRA adapter weights, so embeddings differ across folds even for identical input cells.
2. **Different round $\rightarrow$ different input cells.** With `is_training=True`, each round randomly picks 2,000 different cells from a patient's pool. So embeddings differ across rounds within the same fold.

Consequence: per seed, roughly 202 patients $\times$ 15 passes $\times$ 2,000 cells $\approx$ **6 million cell-level scGPT forward computations**, and 30 million across all 5 seeds.